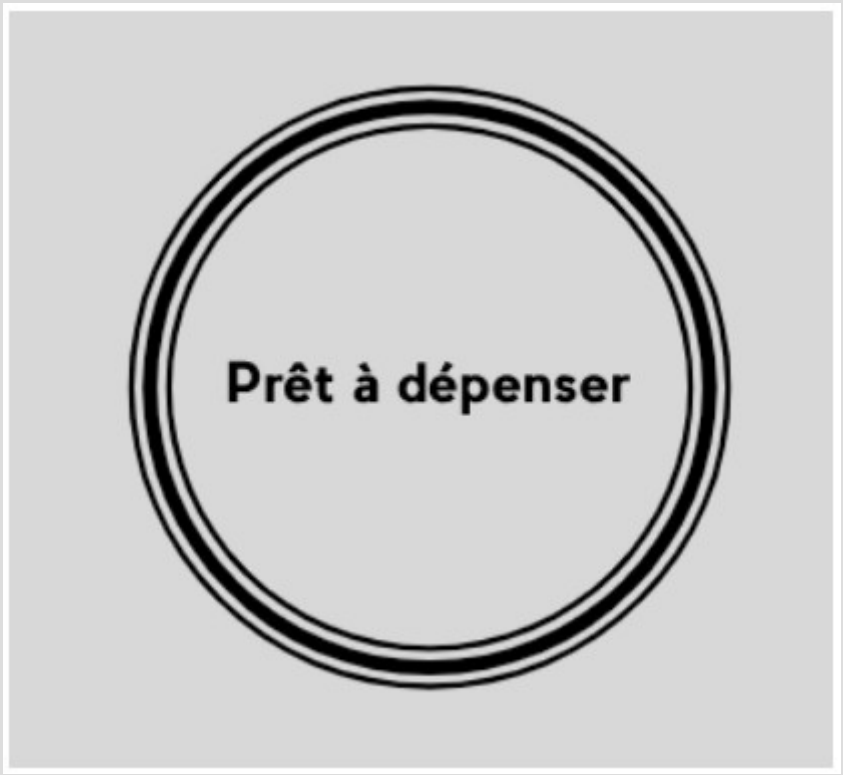
Implémentation d'un modèle de scoring



1. Analyse et modélisation

- L’entreprise souhaite mettre en œuvre un outil de “scoring crédit” pour calculer la probabilité qu’un client rembourse son crédit, puis classifie la demande en crédit accordé ou refusé.
- Elle souhaite donc développer un algorithme de classification en s’appuyant sur des sources de données variées
- Les clients sont de plus en plus demandeurs de transparence vis-à-vis des décisions d’octroi de crédit.

1. Prétraitement des descriptions des produits et des images

- Mission : développer un dashboard interactif pour :
  - expliquer de façon la plus transparente possible les décisions d’octroi de crédit
  - permettre aux clients de disposer de leurs informations personnelles et de les explorer facilement

1. Prétraitement des descriptions des produits et des images

- Dataframe final avec jointures des dataframes fournis : sa taille après analyse exploratoire & nettoyage est de 274705 lignes et 558 colonnes.
- Prétraitement inspiré du kernel Kaggle fourni en ressources : https://www.kaggle.com/code/jsaguiar/lightgbm-with-simple-features/script
- Nous allons passer à la partie modélisation

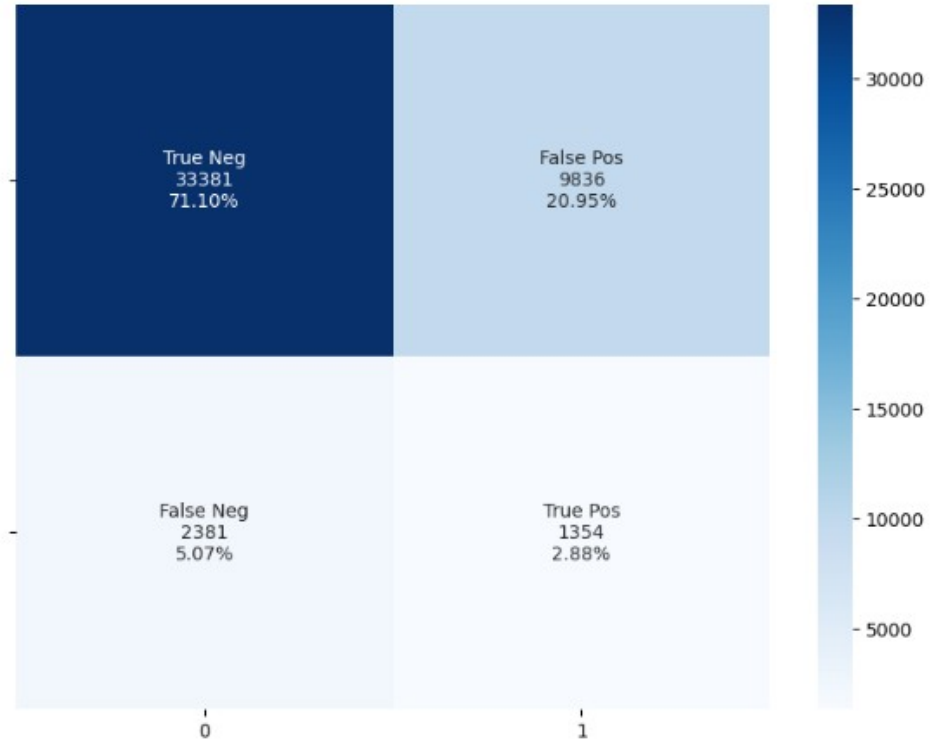
1. Prétraitement des descriptions des produits et des images

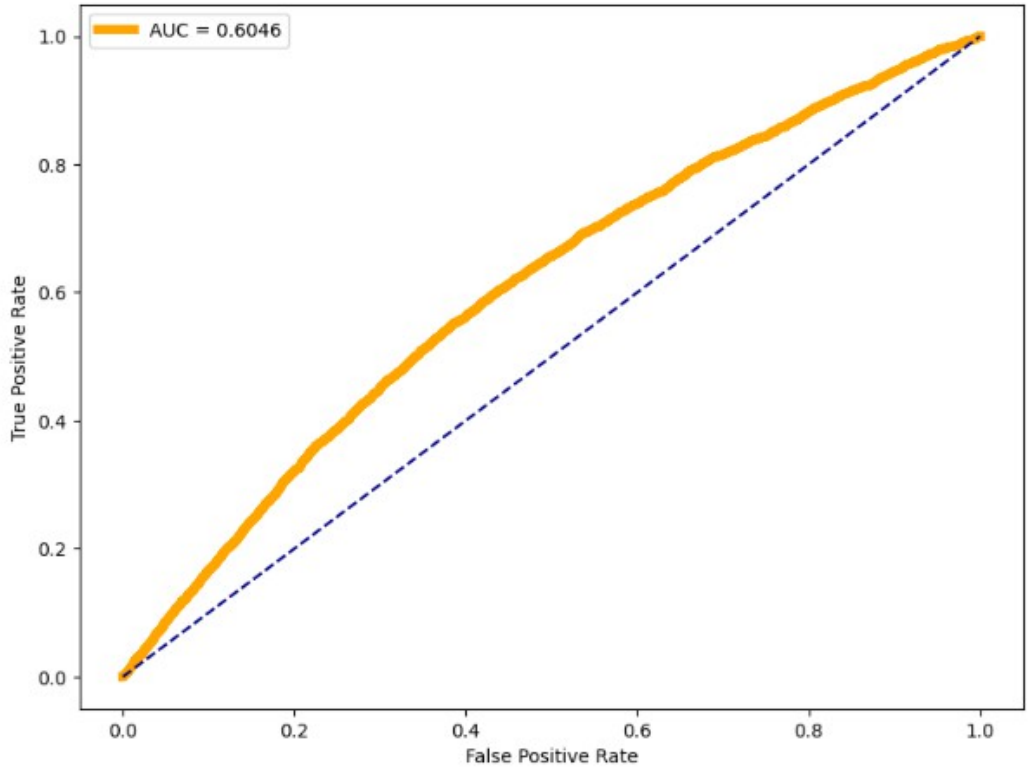
- Deux algorithmes d'apprentissage automatique supervisé seront testés ici :
  - La régression logistique
  - Le Random Forest Classifier

- Les résultats seront optimisés via GridSearchCV.

1. Prétraitement des descriptions des produits et des images

- La régression logistique

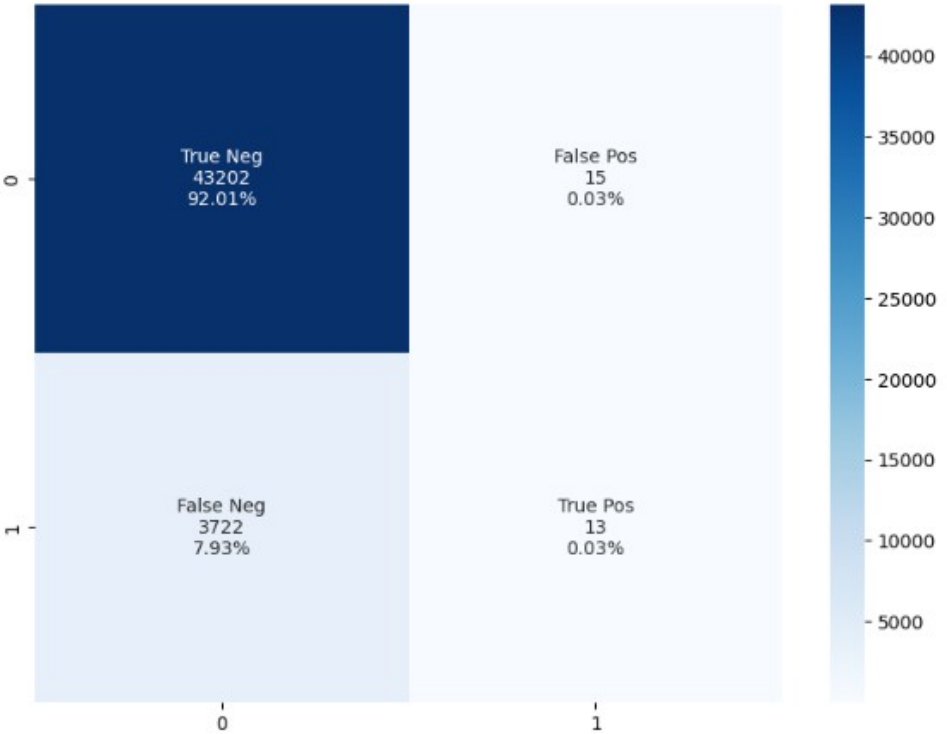


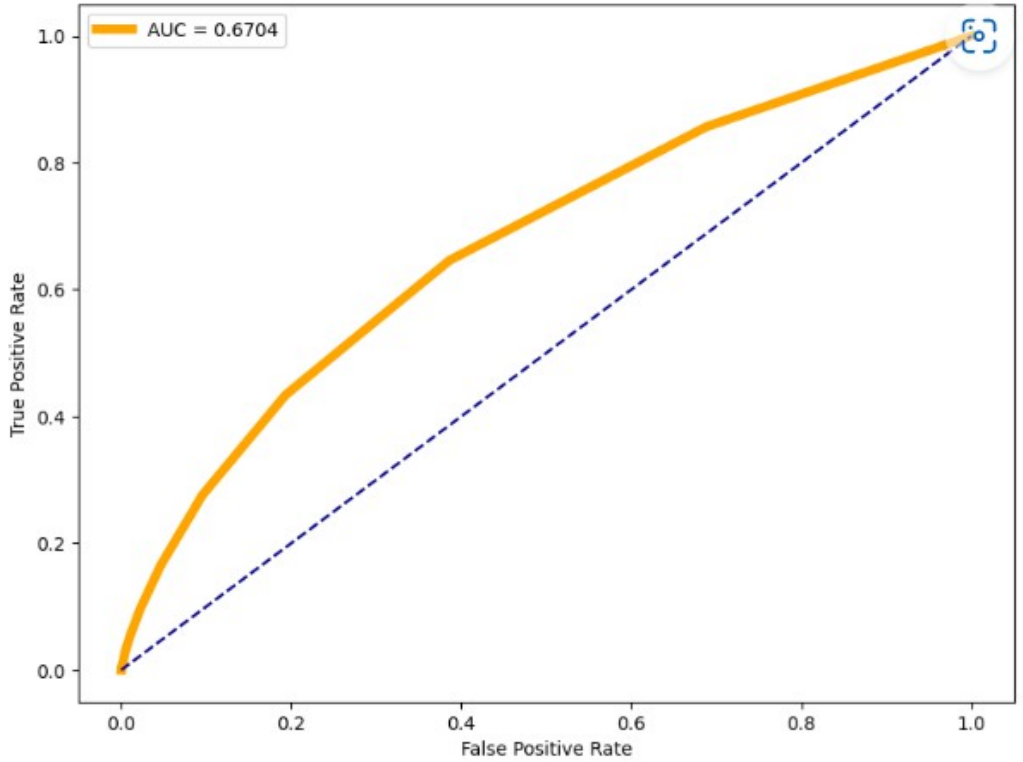


Score AUC : 0,60 et f_beta-score = 0,13 (avec beta = 0,1 et un seuil de proba de 0,6)

1. Prétraitement des descriptions des produits et des images

- Random Forest Classifier





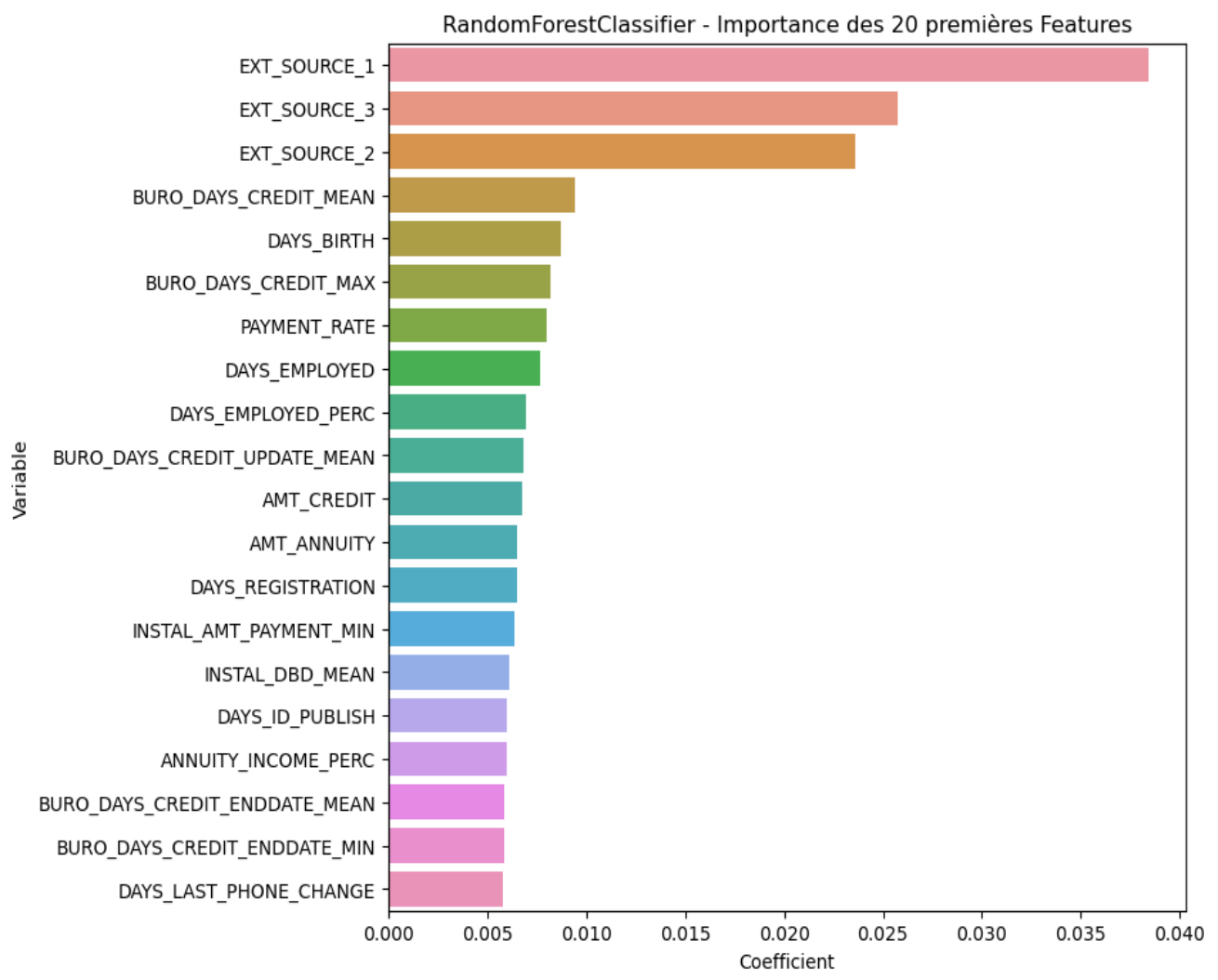
Score AUC : 0,67 et f_beta-score_max = 0,29 (avec beta = 0,1 et seuil proba=0,3)

1. Prétraitement des descriptions des produits et des images

- RandomForestClassifier sera choisi car présente les meilleurs scores ici.
- Remarquons au passage que les bêta-scores restent particulièrement faibles pour les deux modèles.

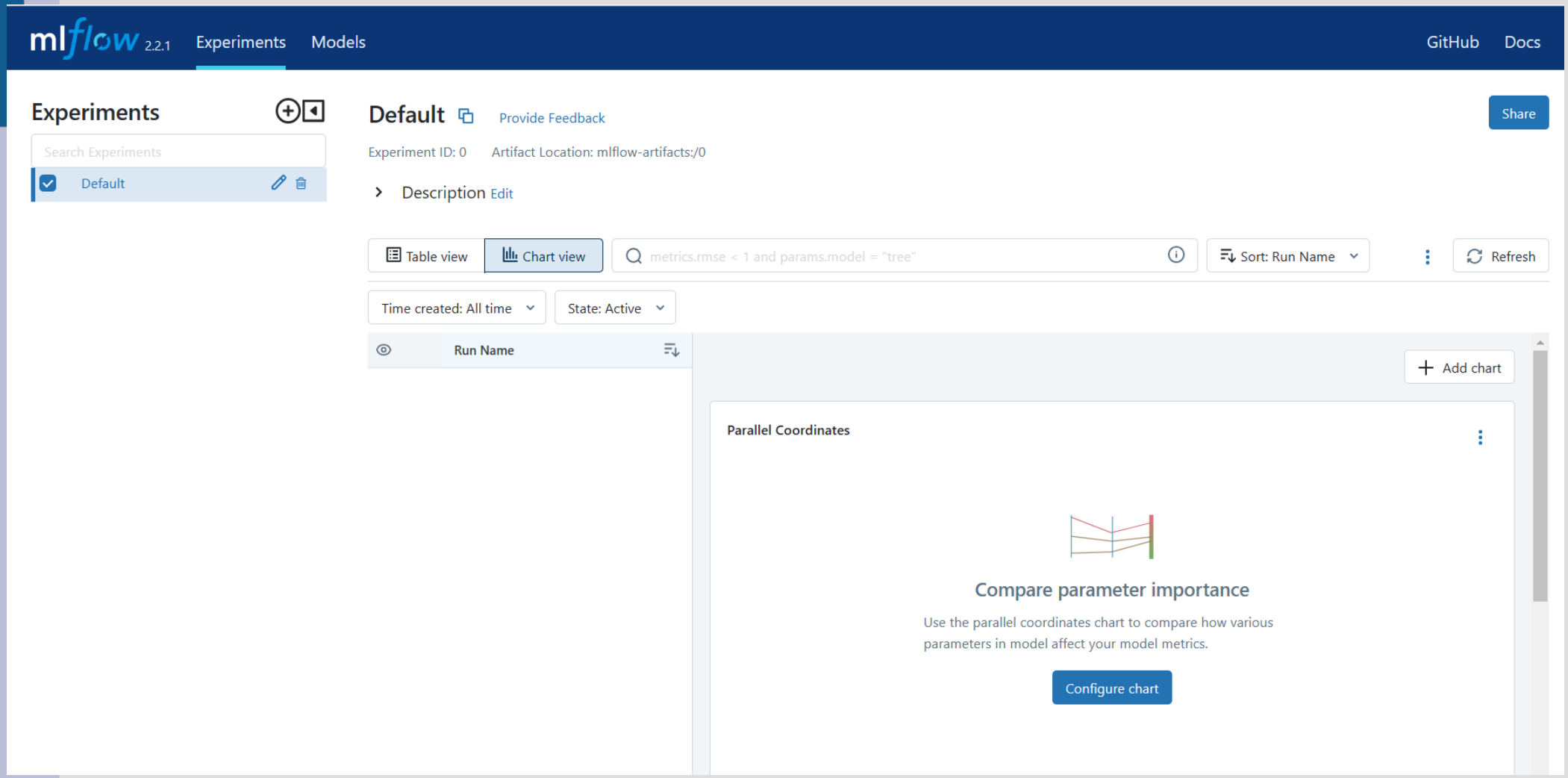
2. Réduction de dimensions

- Elle se fera ici pour le modèle choisi avec le choix des features les plus importantes.



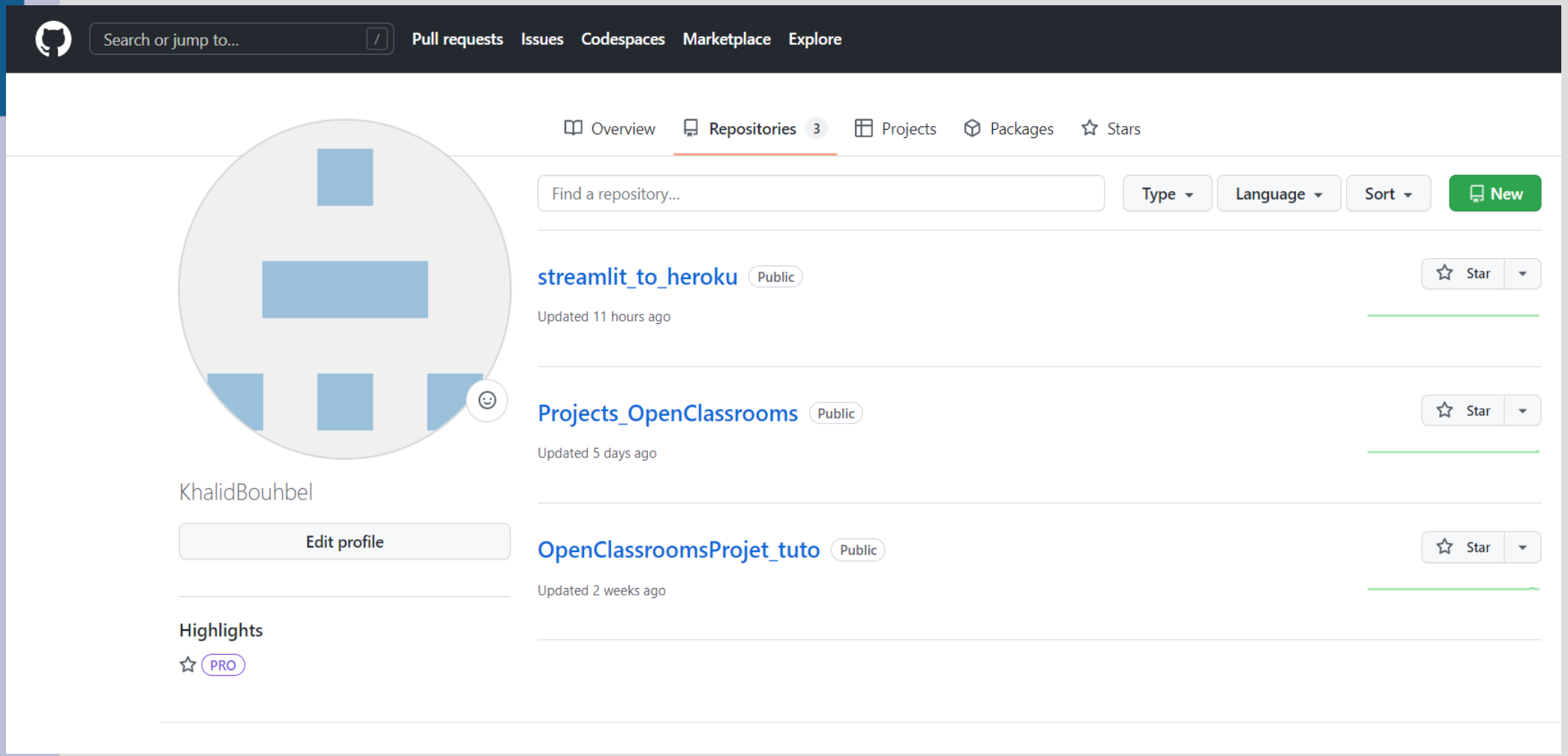
3. Tracking

- Elle se fera ici via MLFlow UI :



4. Pipeline de Déploiement

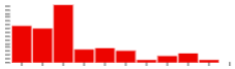
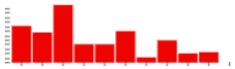
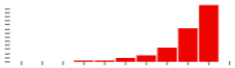
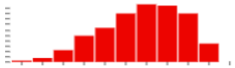
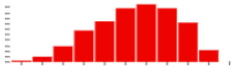
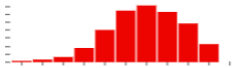
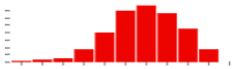
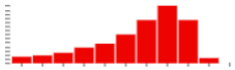
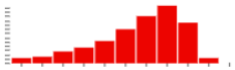
- Elle se fera ici via Git & GitHub : https://github.com/KhalidBouhbel



5. Data Drift & analyses

Drift is detected for 30.00% of features (3 out of 10). Dataset Drift is NOT detected.

Search

Feature	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
TARGET	cat			Detected	Jensen-Shannon distance	0.160053
prediction	num			Detected	Jensen-Shannon distance	0.14084
PAYMENT_RATE	num			Detected	Wasserstein distance (normed)	0.490609
DAYS_EMPLOYED	num			Not Detected	Wasserstein distance (normed)	0.064504
DAYS_BIRTH	num			Not Detected	Wasserstein distance (normed)	0.064497
BURO_DAYS_CREDIT_MEAN	num			Not Detected	Wasserstein distance (normed)	0.054692
EXT_SOURCE_2	num			Not Detected	Wasserstein distance (normed)	0.046852

5. Data drift & analyses

- Le Data Drift ou dérive de donnée survient quand les données sur lequel s’exécute le modèle diffèrent de façon trop importantes des données d'entraînement. Ce problème doit être détecté et anticipé, car il dégrade les performances de prédiction au fur et à mesure du temps.
- Nombreux facteurs :
  - La dimension temporelle
  - La saisonnalité
  - Données collectées avant un événement ne seront plus forcément valables après
- Ici le drift concerne principalement la variable TARGET et TAUX DE PAIEMENT : c'est normal pour la TARGET car c'est la valeur cible, Avoir la même serait synonyme d'overfitting.
  Pour le PAYMENT_RATE : il diffère naturellement en fonction des individus, néanmoins une très grosse différence entre le jeu de données train et test peut fortement impacter nos prévisions.

6. Dashboard déployé sur le cloud

- Démonstration : (ici Heroku fera office de cloud)

Conclusions

- Observations:
  - Pertinence du choix du modèle ? Des paramètres ?
    GridSearchCV : nous avons été limités au niveau des paramètres car jeu de données trop volumineux : algorithmes tournaient pendant des jours !

Merci pour votre écoute !